

HDOC DAY-32 ASSIGNMENT

Menu-Driven Operations on Variable-Length 2D Arrays

1. Problem Understanding

The task is to write a menu-driven C program using separate functions and variable-length 2D arrays. The user selects one of the following operations:

- Prove that matrix addition is commutative using two variable-length 2D arrays.
- Prove that matrix multiplication is not commutative using two variable-length 2D arrays.
- Exit the program.

For addition, both matrices must have the same dimensions. For multiplication in both orders, the dimensions must allow both $A \times B$ and $B \times A$; using two square matrices of the same order keeps the program simple and clearly demonstrates non-commutativity.

2. Standard Test Cases

Test Case	Choice	Order n	Matrix A	Matrix B	Expected Result	Status
TC1	1	2	1 2 3 4	5 6 7 8	Equal sums; commutative	Pass
TC2	1	2	2 0 1 3	4 1 2 5	Equal sums; commutative	Pass
TC3	2	2	1 2 3 4	0 1 1 0	Different products; not commutative	Pass
TC4	2	2	1 1 0 1	1 0 1 1	Different products; not commutative	Pass
TC5	3	—	—	—	Program exits	Pass

3. Algorithm

A. Common Menu Logic

1. Start.
2. Display the menu options.
3. Read the user's choice.
4. Use switch-case to call the required function.
5. If choice is 1, call the matrix-addition function.
6. If choice is 2, call the matrix-multiplication function.
7. If choice is 3, exit the program.
8. For any other choice, display "Invalid choice".
9. Stop.

B. Matrix Addition - Proving Commutativity

1. Read the order n of the two square matrices.
2. Declare two variable-length matrices $A[n][n]$ and $B[n][n]$.
3. Read all elements of matrix A and matrix B.

4. Calculate $A + B$ and store it in sum1.
5. Calculate $B + A$ and store it in sum2.
6. Display both result matrices.
7. Compare corresponding elements of sum1 and sum2.
8. If all elements are equal, display that matrix addition is commutative.
9. Stop.

C. Matrix Multiplication - Proving Non-Commutativity

1. Read the order n of the two square matrices.
2. Declare two variable-length matrices $A[n][n]$ and $B[n][n]$.
3. Read all elements of matrix A and matrix B .
4. Set every element of AB and BA to 0.
5. Calculate $A \times B$ using three nested loops.
6. Calculate $B \times A$ using three nested loops.
7. Display both product matrices.
8. Compare corresponding elements of AB and BA .
9. If at least one pair is different, display that matrix multiplication is not commutative; otherwise display that the selected matrices give equal products.
10. Stop.

4. C Program

```
#include <stdio.h>

void addition()

{
    int n, i, j, same = 1;

    printf("Enter the order of the matrices: ");

    scanf("%d", &n);

    int a[n][n], b[n][n], sum1[n][n], sum2[n][n];

    printf("Enter matrix A:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter matrix B:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &b[i][j]);
        }
    }

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
```

```
    sum1[i][j] = a[i][j] + b[i][j];
    sum2[i][j] = b[i][j] + a[i][j];
}
}
printf("A + B:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", sum1[i][j]);
    }
    printf("\n");
}
printf("B + A:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", sum2[i][j]);
        if (sum1[i][j] != sum2[i][j])
        {
            same = 0;
        }
    }
    printf("\n");
}
if (same == 1)
{
    printf("Matrix addition is commutative.\n");
}
```

```

    }
}
void multiplication()
{
    int n, i, j, k, same = 1;
    printf("Enter the order of the matrices: ");
    scanf("%d", &n);
    int a[n][n], b[n][n], ab[n][n], ba[n][n];
    printf("Enter matrix A:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    printf("Enter matrix B:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &b[i][j]);
        }
    }
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            ab[i][j] = 0;

```

```

    ba[i][j] = 0;
    for (k = 0; k < n; k++)
    {
        ab[i][j] += a[i][k] * b[k][j];
        ba[i][j] += b[i][k] * a[k][j];
    }
}

}

printf("A x B:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", ab[i][j]);
    }
    printf("\n");
}

printf("B x A:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", ba[i][j]);
        if (ab[i][j] != ba[i][j])
        {
            same = 0;
        }
    }
}

printf("\n");

```

```
}  
if (same == 0)  
{  
    printf("Matrix multiplication is not commutative.\n");  
}  
else  
{  
    printf("For these matrices, A x B and B x A are equal.\n");  
}  
}  
int main()  
{  
    int choice;  
  
    do  
    {  
        printf("\nMENU\n");  
        printf("1. Matrix addition\n");  
        printf("2. Matrix multiplication\n");  
        printf("3. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
        switch (choice)  
        {  
            case 1:  
                addition();  
                break;  
            case 2:  
                multiplication();
```

```

        break;

    case 3:

        printf("Program ended.\n");

        break;

    default:

        printf("Invalid choice.\n");

    }

}

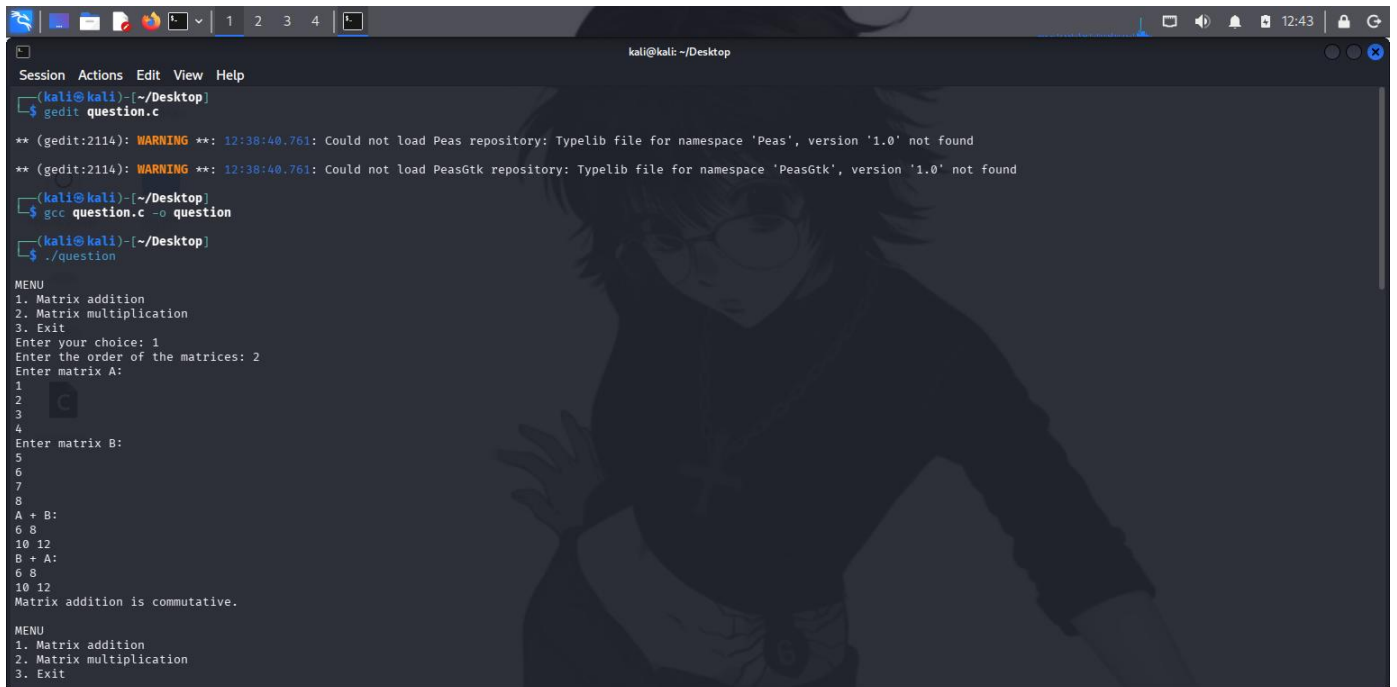
while (choice != 3);

return 0;

}

```

5. Compilation and Execution Evidence



The screenshot shows a Kali Linux terminal window with the following content:

```

kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)~[~/Desktop]
$ gedit question.c

** (gedit:2114): WARNING **: 12:38:40.761: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2114): WARNING **: 12:38:40.761: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~[~/Desktop]
$ gcc question.c -o question

(kali@kali)~[~/Desktop]
$ ./question

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 1
Enter the order of the matrices: 2
Enter matrix A:
1
2
3
4
Enter matrix B:
5
6
7
8
A + B:
6 8
10 12
B + A:
6 8
10 12
Matrix addition is commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit

```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 1
Enter the order of the matrices: 2
Enter matrix A:
1
0
1
3
Enter matrix B:
4
1
2
5
A + B:
5 1
3 8
B + A:
5 1
3 8
Matrix addition is commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 2
Enter the order of the matrices: 2
Enter matrix A:
1
2
3
4
Enter matrix B:
0
1
1
0
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

Enter matrix B:
0
1
1
0
A x B:
2 1
4 3
B x A:
3 4
1 2
Matrix multiplication is not commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 2
Enter the order of the matrices: 2
Enter matrix A:
1
1
0
1
Enter matrix B:
1
0
1
1
A x B:
2 1
1 1
B x A:
1 1
1 2
Matrix multiplication is not commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
2 1
4 3
B x A:
3 4
1 2
Matrix multiplication is not commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 2
Enter the order of the matrices: 2
Enter matrix A:
1 1
1 0
1 1
Enter matrix B:
1 1
1 0
1 1
A x B:
2 1
2 1
1 1
B x A:
1 1
1 2
Matrix multiplication is not commutative.

MENU
1. Matrix addition
2. Matrix multiplication
3. Exit
Enter your choice: 3
Program ended.
kali@kali) [~/Desktop]
```

6. Verification of All Test Cases

Test Case	Choice	Order n	Matrix A	Matrix B	Expected Result	Actual Result	Status
TC1	1	2	1 2 3 4	5 6 7 8	Equal sums; commutative	Equal sums; commutative	Pass
TC2	1	2	2 0 1 3	4 1 2 5	Equal sums; commutative	Equal sums; commutative	Pass
TC3	2	2	1 2 3 4	0 1 1 0	Different products; not commutative	Different products; not commutative	Pass
TC4	2	2	1 1 0 1	1 0 1 1	Different products; not commutative	Different products; not commutative	Pass
TC5	3	—	—	—	Program exits	Program exits	Pass